

Informe: Proyecto Final

Materia: Programación Avanzada

Integrantes: Lionel Gutierrez, Ignacio Romero, Nahuel Leyes

Año: 2026

Profesores: Juan Gonzales y Javier Rosenstein

Introducción

El proyecto consiste en el desarrollo de una aplicación de mensajería en tiempo real similar a WhatsApp, implementada completamente en Java.

El objetivo principal fue integrar los conceptos fundamentales vistos durante el cursado de la materia: Programación Orientada a Objetos (POO), Arquitectura Cliente-Servidor, Patrones de Diseño y Arquitectura, manejo de Hilos, Sockets y la biblioteca grafica Swing de Java.

Descripción del Proyecto

La aplicación permite que múltiples clientes se conecten a un servidor central e intercambien mensajes tanto de forma grupal como privada. Al iniciar el cliente, se presenta una pantalla de login donde el usuario ingresa su nombre y correo. Una vez autenticado, accede a la ventana principal del chat donde puede visualizar todos los usuarios conectados y comunicarse con ellos.

Funcionalidades Principales

- Login con validación de nombre (obligatorio) y correo (opcional)
- Chat grupal visible para todos los usuarios conectados
- Chat privado entre dos usuarios específicos
- Lista de usuarios dinámica que se actualiza en tiempo real
- Contador cantidad de usuarios conectados
- Hora actual en tiempo real y marca de tiempo en cada mensaje
- Detección automática de desconexión de los usuarios

Conceptos Aplicados

Programación Orientada a Objetos (POO)

El proyecto hace uso extensivo de POO en la definición de clases con atributos privados, los métodos getters y setters, constructores parametrizados y encapsulamiento. Las clases principales son: Modelo, Vista, Login, Controlador, Cliente, ClienteR, Servidor y ClienteHilo.

Patrón de Arquitectura MVC

Se implemento el patrón Modelo-Vista-Controlador para separar responsabilidades:

- Modelo: almacena el nombre y correo del usuario
- Vista: interfaz gráfica con Swing (Login y ventana de chat)
- Controlador: coordina la interacción entre Modelo y Vista, y gestiona los eventos.

Patrón de diseño: Singleton

Se aplicó el patrón Singleton en la clase Servidor para garantizar que solo exista una única instancia del servidor en ejecución. Esto se logra mediante un constructor privado y un método estático getInstance() que es el que controla la creación del objeto.

Arquitectura C/S

La arquitectura C/S se implementa con el Servidor escuchando en el puerto 5000 mediante un ServerSocket. Por cada cliente que se conecta, se crea un hilo independiente (ClienteHilo) que gestiona la comunicación con ese cliente específico, permitiendo múltiples conexiones simultáneas.

Hilos (Threads)

Se utilizan hilos en dos lugares clave:

- ClienteHilo (servidor): atiende a cada cliente conectado en un hilo separado, implementando la interfaz Runnable
- ClienteR (cliente): hilo receptor que escucha mensajes entrantes del servidor de forma asíncrona sin bloquear la interfaz gráfica

Sockets

La comunicación en red se realiza mediante Socket y ServerSocket de Java. Los datos se transmiten usando DataInputStream y DataOutputStream con el método writeUTF/readUTF para enviar y recibir cadenas de texto de forma eficiente.

Manejo de Excepciones

Se utiliza try-catch en todas las operaciones de red para manejar desconexiones inesperadas, errores de E/S y situaciones de red inestable. Cuando un cliente se desconecta abruptamente, el servidor detecta la excepción, elimina al cliente de las listas y notifica a los demás usuarios.

Swing

La interfaz gráfica se construyó con Java Swing usando el diseñador visual de NetBeans. Se utilizaron componentes como JFrame, JPanel, JList con DefaultListModel, JTextArea, JTextField, JButton, JScrollPane y javax.swing.Timer para la actualización de la hora en tiempo real.

Arquitectura del Proyecto

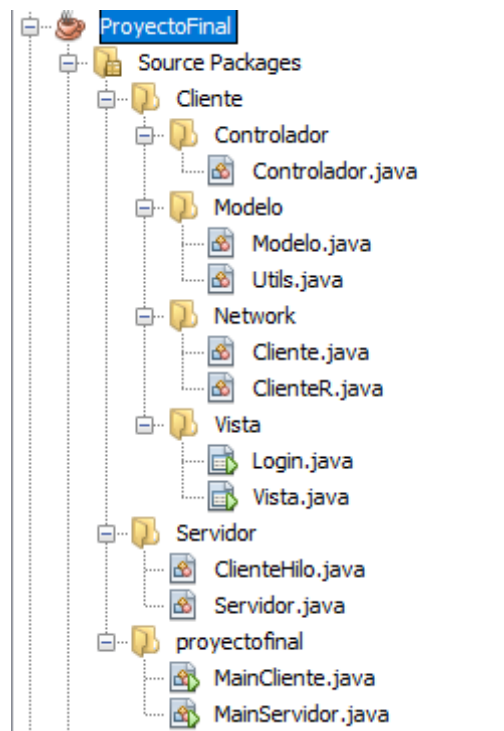
El sistema se organiza en dos partes separadas dentro del mismo proyecto:

Lado Servidor

- MainServidor: punto de entrada, obtiene la instancia del Servidor vía Singleton.
- Servidor: clase principal con patrón Singleton, mantiene las listas de clientes conectados.
- ClienteHilo: hilo que atiende a cada cliente, gestiona mensajes grupales y privados.

Lado Cliente

- MainCliente: punto de entrada, instancia Modelo, Login y Controlador
- Modelo: almacena nombre (obligatorio) y correo (opcional) del usuario autenticado
- Login (Vista): ventana de inicio de sesión con validación
- Vista: ventana principal del chat con lista de usuarios y área de mensajes
- Controlador: implementa ActionListener, coordina toda la lógica
- Cliente (Network): gestiona la conexión socket y el envío de mensajes
- ClienteR (Network): hilo receptor de mensajes entrantes
- Utils: clase utilitaria con métodos de validación y extras.



Flujo del Proyecto

1. Se ejecuta el Servidor: aparece 'Servidor iniciado' en la consola.
2. Se ejecuta el Cliente: aparece la ventana de Login.
3. El usuario completa nombre (obligatorio) y el correo (opcional) con las validaciones correspondientes.
4. Al presionar 'Conectarse': se establece la conexión socket, se cierra el Login y se abre la ventana del chat.
5. El servidor notifica a todos los clientes sobre el nuevo usuario conectado.
6. El usuario puede seleccionar 'Grupo General' para enviar mensajes a todos, o un usuario específico para un chat privado.
7. Al cerrar la ventana, el servidor detecta la desconexión y notifica a los demás usuarios.

Repositorio

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de Github:

<https://github.com/IgnaaRo/JavaProyectoFinal-Programacion-Avanzada.git>